DSPy + Context Engineering Basics to Pro — Complete Course

Based on the YouTube course by Avishek Biswas (Neural Breakdown with AVB)

youtu.be/5Bym0ffALaU • github.com/avbiswas/context-engineering-dspy

#	Topic	What You'll Learn
1	What is Context Engineering?	The big picture — why prompts alone aren't enough
2	Why DSPy?	Structured I/O, auto-retry, declarative design
3	Atomic Prompts (Level 1)	dspy.Predict & dspy.ChainOfThought basics
4	Multi-Step Workflows (Level 2)	Sequential, iterative, conditional, reflection
5	Evaluation (Level 3)	MLflow tracking, pairwise ranking, metrics
6	Tool Calling (Level 4)	dspy.ReAct, external APIs, file I/O, MCP
7	RAG — Retrieval (Level 5)	Embeddings, BM25, HyDE, hybrid search, memory
8	Production Best Practices	Evaluation, structured outputs, monitoring

01 What is Context Engineering?

Context Engineering is the art and science of deciding **what information** to put into an LLM's context window — and **when** — so the model can successfully complete a complex task. It goes far beyond writing a single prompt.

The term was popularised by Andrej Karpathy. His key insight: most real-world LLM tasks are too complex for a single prompt. You need to break them into sub-problems, route them through specialised agents, and carefully control what each agent sees.

The Old Way (Simple Prompting)	Context Engineering Approach
One giant prompt with all information	Break the task into sub-tasks for specialised agents
LLM tries to do everything at once	Each agent gets exactly the context it needs
Hard to debug or improve	Modular — each piece can be tested independently
Sensitive to prompt wording	Robust — uses structured inputs/outputs and retries

■■ Key Insight: Stuffing everything into the LLM's context leads to 'context rot' — too much irrelevant information degrades quality and causes hallucinations.

■ The 7 Pillars of Context Engineering

1. **Modular sub-tasks:** Break large problems into smaller, focused tasks per agent.
2. **Appropriate agents:** Use smaller/cheaper models for simple tasks, powerful ones for complex.
3. **Intermediate tokens:** Allow agents to generate reasoning steps before final answers.
4. **Control flow:** Connect agents with sequential, parallel, or conditional logic.
5. **External knowledge:** Retrieve information from databases (RAG) or web search (tools).
6. **Actions:** Let agents call tools (web search, file I/O, APIs).
7. **Evaluation:** Measure output quality with metrics and track with observability tools.

02 Why DSPy?

DSPy is a **declarative framework** for building LLM applications. Instead of writing raw prompts as strings, you define **Signatures** (what goes in, what comes out) and **Modules** (how to transform them). DSPy handles prompt generation, JSON parsing, and error recovery automatically.

Feature	Raw API (OpenAI)	DSPy
Structured output	Parse JSON yourself (fragile)	Auto-validated Pydantic objects
Error handling	Your code crashes on bad JSON	Auto-retries with corrective prompt
Chain of Thought	Write extra parsing logic	Just use dspy.ChainOfThought
Multi-model routing	Manual if/else logic	set_lm() per module
Readability	Giant string prompt	Clean Python class definition

■ DSPy separates WHAT (Signature = input/output contract) from HOW (Module = control flow logic). This makes code reusable, testable, and easy to extend.

03 Level 1 — Atomic Prompts

An **atomic prompt** is a single question or task sent to the LLM. This is where everything starts. DSPy makes these robust by auto-handling structured outputs and retries.

■ dspy.Predict — Basic Module

The simplest DSPy module. You define a Signature (inputs + outputs) and pass it to **dspy.Predict**. DSPy builds the prompt, calls the LLM, and returns a typed result object.

```
import dspy

# Step 1: Define a Signature — what goes IN, what comes OUT
class JokeGenerator(dspy.Signature):
"""You are a comedian who tells jokes. You are always funny."""
query: str = dspy.InputField() # Input: user's request
setup: str = dspy.OutputField() # Output 1: joke setup
punchline: str = dspy.OutputField() # Output 2: punchline
delivery: str = dspy.OutputField() # Output 3: full comedian delivery

# Step 2: Create the module and set the language model
joke_gen = dspy.Predict(JokeGenerator)
joke_gen.set_lm(lm=dspy.LM("openai/gpt-4.1-mini", temperature=1))

# Step 3: Call it — outputs are typed, auto-validated
result = joke_gen(query="Write a joke about AI")

print(result.setup) # e.g. "Why did the AI go to therapy?"
print(result.punchline) # e.g. "It had too many unresolved issues."
print(result.delivery) # Full comedian-style delivery
```

dspy.Predict — basic structured output with automatic JSON parsing

■ dspy.ChainOfThought — Reasoning Before Answering

Chain of Thought is a key context engineering technique. The LLM generates *reasoning text* before producing the final answer. This populates the context with self-generated thoughts, leading to better answers on complex problems. In DSPy, you just swap one word.

```
# Exact same Signature as before — just swap dspy.Predict for
dspy.ChainOfThought
joke_gen_cot = dspy.ChainOfThought(JokeGenerator)
joke_gen_cot.set_lm(lm=dspy.LM("openai/gpt-4.1-mini", temperature=1))

result = joke_gen_cot(query="Write a joke about AI")

# Now the result also contains the model's reasoning steps
print(result.reasoning) # The LLM's thought process
print(result.punchline) # The final punchline (usually better!)
```

dspy.ChainOfThought — one-word swap to get reasoning-enhanced outputs

■ Why does Chain of Thought help? The LLM sees its own reasoning in the context window before generating the final answer — this acts like 'thinking out loud', catching mistakes early.

04 Level 2 — Multi-Step Workflows

Real applications need multiple LLM calls connected together. DSPy's **Module** class lets you build custom pipelines with full Python control flow — loops, conditionals, parallel calls, and more.

■ 4a — Sequential Flow

Break a big task into two or more sub-tasks done in sequence. The output of Agent 1 becomes the input of Agent 2. You can assign different LLMs to each step based on complexity.

```
from pydantic import BaseModel

# Intermediate data structure shared between agents
class JokeIdea(BaseModel):
setup: str
contradiction: str
punchline: str

# Agent 1: Generate the idea (cheaper model is fine)
class QueryToIdea(dspy.Signature):
"""Generate a joke idea with setup, contradiction, and punchline."""
query = dspy.InputField()
joke_idea: JokeIdea = dspy.OutputField()

# Agent 2: Turn idea into a full delivery (use stronger model)
class IdeaToJoke(dspy.Signature):
"""Convert a joke idea into a full comedian delivery."""
joke_idea: JokeIdea = dspy.InputField()
joke = dspy.OutputField()

# Custom Module that chains the two agents
class JokeGenerator(dspy.Module):
def __init__(self):
self.query_to_idea = dspy.Predict(QueryToIdea)
self.idea_to_joke = dspy.Predict(IdeaToJoke)
# Use cheaper model for idea, stronger model for delivery
self.query_to_idea.set_lm(dspy.LM("openai/gpt-4.1-mini"))
self.idea_to_joke.set_lm(dspy.LM("openai/gpt-4.1"))

def forward(self, query):
idea = self.query_to_idea(query=query) # Step 1
joke = self.idea_to_joke(joke_idea=idea.joke_idea) # Step 2
return joke

# Use it
generator = JokeGenerator()
result = generator(query="Write a joke about Python programming")
print(result.joke)
```

Sequential pipeline: cheap model for ideation, powerful model for delivery

■ 4b — Iterative Refinement

The LLM generates a draft, a Refinement agent critiques it and gives feedback, and the original LLM improves its output. This loop can run multiple times until quality is satisfactory.

```
class RefinementAgent(dspy.Signature):
"""Review a joke and provide specific feedback to improve it."""
joke: str = dspy.InputField()
feedback: str = dspy.OutputField(
description="Specific improvements: timing, word choice, punchline
delivery"
)

class JokeRefiner(dspy.Signature):
"""Improve the joke using the given feedback."""
original_joke: str = dspy.InputField()
feedback: str = dspy.InputField()
improved_joke: str = dspy.OutputField()

class IterativeJokeGenerator(dspy.Module):
def __init__(self, n_iterations=3):
self.n = n_iterations
self.generator = dspy.Predict(IdeaToJoke)
self.critic = dspy.Predict(RefinementAgent)
self.refiner = dspy.Predict(JokeRefiner)

def forward(self, query):
idea = dspy.Predict(QueryToIdea)(query=query)
joke = self.generator(joke_idea=idea.joke_idea).joke

for i in range(self.n):
feedback = self.critic(joke=joke).feedback
joke = self.refiner(
original_joke=joke,
feedback=feedback
).improved_joke
print(f"Iteration {i+1} feedback: {feedback[:80]}...")

return joke
```

Iterative refinement: the LLM critiques and improves its own output in a loop

■ 4c — Conditional Branching and Parallel Generation

Generate multiple candidates **in parallel** (using async), then use a Judge agent to pick the best one. This is similar to 'best-of-N' sampling.

```
import asyncio

num_samples = 5

class JokeJudge(dspy.Signature):
"""Given several joke ideas, pick the funniest one."""
joke_ideas: list[JokeIdea] = dspy.InputField()
best_idx: int = dspy.OutputField(
description=f"Index (1-{num_samples}) of the funniest joke"
)

class BestJokeGenerator(dspy.Module):
def __init__(self):
self.query_to_idea = dspy.ChainOfThought(QueryToIdea)
self.judge = dspy.ChainOfThought(JokeJudge)
self.idea_to_joke = dspy.ChainOfThought(IdeaToJoke)

async def forward(self, query):
# Generate N ideas in parallel — much faster than sequential!
ideas = await asyncio.gather(*[
self.query_to_idea.acall(query=query)
for _ in range(num_samples)
])

# Judge picks the best
best_idx = (await self.judge.acall(joke_ideas=ideas)).best_idx

# Generate final joke from the best idea
best_idea = ideas[best_idx - 1]
return await self.idea_to_joke.acall(joke_idea=best_idea)

# Run async
result = asyncio.run(BestJokeGenerator().forward("AI and robotics"))
print(result.joke)
```

Parallel generation + LLM judge: generates N candidates, picks the best

■ 4d — Reflection

Reflection is a pattern where the agent examines its own previous output and decides whether it is good enough, or needs to try again with a different approach.

```
class JokeReflector(dspy.Signature):
"""Evaluate the joke quality and decide if it needs improvement."""
joke: str = dspy.InputField()
is_funny: bool = dspy.OutputField(description="True if the joke is good")
reason: str = dspy.OutputField(description="Why the joke is or isn't
funny")

class ReflectiveJokeGenerator(dspy.Module):
def __init__(self, max_attempts=3):
self.generator = dspy.Predict(IdeaToJoke)
self.reflector = dspy.Predict(JokeReflector)
self.max_attempts = max_attempts

def forward(self, query):
idea = dspy.Predict(QueryToIdea)(query=query)
for attempt in range(self.max_attempts):
result = self.generator(joke_idea=idea.joke_idea)
review = self.reflector(joke=result.joke)
if review.is_funny:
print(f"Accepted on attempt {attempt + 1}")
return result.joke
print(f"Attempt {attempt+1} rejected: {review.reason}")
return result.joke # Return best effort after max attempts
```

Reflection: the agent evaluates its own output and retries if quality is insufficient

05 Level 3 — Evaluation & Monitoring

LLM outputs are probabilistic — the same prompt can give different results each run. You need **evaluation** to know which configuration works best, and **monitoring** to track performance over time.

■ MLflow Integration

```
import mlflow

# Start the MLflow server first (run in terminal):
# mlflow server --backend-store-uri sqlite:///mydb.sqlite --port 5000

# Then in your Python code:
mlflow.autolog() # Auto-tracks all DSPy calls
mlflow.set_tracking_uri("http://127.0.0.1:5000")
mlflow.set_experiment("joke-generation-experiment")

# Now run your DSPy program normally
generator = JokeGenerator()
result = generator(query="Write a joke about Python")

# MLflow automatically records:
# - The full prompt sent to the LLM
# - The LLM response
# - Token counts and cost
# - Latency per module
# - Any errors or retries

# Visit http://localhost:5000 to see the dashboard
```

MLflow auto-logging: tracks every LLM call, token usage, latency, and cost

■ Pairwise Evaluation (LLM-as-Judge)

When you have no ground-truth labels, use an LLM to compare two outputs and pick which is better. Run many such comparisons to get a reliable ranking.

```
class PairwiseJudge(dspy.Signature):
"""Compare two jokes and decide which is funnier."""
joke_a: str = dspy.InputField()
joke_b: str = dspy.InputField()
winner: str = dspy.OutputField(
description="Either 'A' or 'B' — the funnier joke"
)
reasoning: str = dspy.OutputField()

judge = dspy.ChainOfThought(PairwiseJudge)
judge.set_lm(dspy.LM("openai/gpt-4.1")) # Use strongest model as judge

# Compare two outputs
result = judge(joke_a="Setup A... Punchline A", joke_b="Setup B... Punchline
B")
print(f"Winner: Joke {result.winner}")
print(f"Reason: {result.reasoning}")
```

Pairwise LLM judging: use a strong model to compare and rank outputs

06 Level 4 — Tool Calling with dspy.ReAct

Tools let the LLM interact with the outside world — search the web, read files, call APIs. **dspy.ReAct** (Reasoning + Acting) automatically decides when to call which tool and incorporates results into the response.

■ Defining a Tool Function

A tool is just a regular Python function with a clear docstring. The docstring tells the LLM *what the tool does*; the type hints tell it *what inputs to pass*.

```
import os
from tavily import TavilyClient

tavily_client = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))

def fetch_recent_news(query: str) -> list[str]:
"""
Search the web for recent news articles (last 7 days).
Returns a list of article content strings.

Args:
query: The search query to find news about
"""
response = tavily_client.search(
query,
search_depth="advanced",
topic="news",
days=7,
max_results=3
)
return [article["content"] for article in response["results"]]

# Test the tool directly
articles = fetch_recent_news("OpenAI GPT-5")
print(articles[0][:200]) # Preview first article
```

Tool definition: any Python function with a clear docstring becomes an LLM tool

■ Using dspy.ReAct — Reasoning + Acting

Pass your Signature and tools to **dspy.ReAct**. The agent will automatically: (1) reason about what info it needs, (2) call the right tool, (3) read the result, (4) decide if it needs more info or can answer now.

```
class NewsHaikuGenerator(dspy.Signature):
"""
Search for the latest news on a topic, then write a haiku about it.
"""
query: str = dspy.InputField()
news_summary: str = dspy.OutputField(
desc="A 2-3 sentence summary of the latest news"
)
haiku: str = dspy.OutputField(
desc="A 3-line haiku (5-7-5 syllables) about the news"
)

# Create the ReAct agent with our tool
agent = dspy.ReAct(
signature=NewsHaikuGenerator,
tools=[fetch_recent_news], # List all available tools
max_iters=3 # Max tool calls before finishing
)
agent.set_lm(dspy.LM("openai/gpt-4.1", temperature=0.7))

# The agent automatically decides to call fetch_recent_news
result = agent(query="artificial intelligence breakthroughs")
print("Summary:", result.news_summary)
print("Haiku:", result.haiku)

# Behind the scenes, the agent does:
# 1. Thought: "I need recent news to write an accurate haiku"
# 2. Action: fetch_recent_news("artificial intelligence breakthroughs")
# 3. Observation: [article contents...]
# 4. Thought: "I have enough info now"
# 5. Answer: generates summary and haiku
```

dspy.ReAct: the agent reasons, calls tools, observes results, then answers

■ MCP Servers — The New Standard for Agent Tools

Model Context Protocol (MCP) is an emerging industry standard for serving tools to LLMs. It uses a Client-Server model: the LLM (client) sends requests to an MCP server which executes actions and returns results.

Approach	Best For	Example
Plain Python function	Simple, single-purpose tools	fetch_recent_news()
MCP Server	Shared tools used by many agents	File system, database, browser
Tool calling + file I/O	Complex agent environments	Gemini CLI, Claude Code

07 Level 5 — Retrieval-Augmented Generation (RAG)

RAG lets the LLM answer questions using **your own data**, not just its training knowledge. It retrieves relevant chunks of text from a database and injects them into the LLM's context.

■ Basic RAG Pipeline

```
import dspy
import numpy as np

# ■■ PREPROCESSING PHASE (done once) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# 1. Load your documents
documents = ["Doc 1 text...", "Doc 2 text...", "Doc 3 text..."]

# 2. Embed each document into a vector
from openai import OpenAI
client = OpenAI()

def embed(text: str) -> list[float]:
return client.embeddings.create(
input=text, model="text-embedding-3-small"
).data[0].embedding

doc_embeddings = [embed(doc) for doc in documents]

# ■■ INFERENCE PHASE (runs per query) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
def retrieve(query: str, top_k: int = 3) -> list[str]:
"""Find the most relevant documents for the query."""
query_emb = embed(query)
# Cosine similarity between query and all documents
scores = [
np.dot(query_emb, doc_emb) for doc_emb in doc_embeddings
]
# Return top-k most similar documents
top_indices = np.argsort(scores)[-top_k:][::-1]
return [documents[i] for i in top_indices]

class RAGAnswer(dspy.Signature):
"""Answer the question using only the provided context."""
question: str = dspy.InputField()
context: str = dspy.InputField(
desc="Relevant documents retrieved from the database"
)
answer: str = dspy.OutputField()

def rag_answer(question: str) -> str:
docs = retrieve(question)
context = "\n\n".join(docs)
result = dspy.Predict(RAGAnswer)(question=question, context=context)
return result.answer
```

Basic RAG: embed documents offline, retrieve similar ones at query time, then generate

■ Query Rewriting — Better Retrieval Queries

Users write messy, conversational queries. These often don't match the vocabulary in your database. **Query rewriting** transforms the user's query into one optimized for retrieval.

```
class QueryRewriting(dspy.Signature):
"""
Rewrite the user query to improve document retrieval accuracy.
Fix grammar, add keywords, and contextualize with conversation history.
"""
user_query: str = dspy.InputField()
conversation: str = dspy.InputField(
description="Recent conversation history for context"
)
rewritten_query: str = dspy.OutputField(
description="Optimized query for semantic search"
)

rewriter = dspy.Predict(QueryRewriting)

# Example: messy conversational query
raw_query = "what about its pricing?" # "its" = unclear reference

# With conversation history, the LLM clarifies the reference
result = rewriter(
user_query=raw_query,
conversation="User: Tell me about GPT-4. Assistant: GPT-4 is..."
)
print(result.rewritten_query)
# "What is the pricing and cost structure of GPT-4?"
```

Query rewriting: fix vague pronouns, add keywords, improve retrieval accuracy

■ HyDE — Hypothetical Document Embeddings

HyDE is a clever trick: instead of embedding the user's short question, ask the LLM to write a *hypothetical answer* first, then embed that. The hypothetical answer contains the vocabulary and concepts that a real answer would contain — making it match the document database much better.

```
class HypotheticalAnswer(dspy.Signature):
"""
Generate a hypothetical answer to this question as if you knew the answer.
This will be used to search a document database.
"""
question: str = dspy.InputField()
hypothetical_doc: str = dspy.OutputField(
description="A paragraph that would answer this question, "
"using technical vocabulary from the domain"
)

hyde_generator = dspy.Predict(HypotheticalAnswer)

class HyDERetriever:
def retrieve(self, question: str, top_k: int = 3):
# Step 1: Generate hypothetical answer
hyp = hyde_generator(question=question)

# Step 2: Embed the hypothetical answer (not the question!)
hyp_embedding = embed(hyp.hypothetical_doc)

# Step 3: Find real documents closest to the hypothetical answer
scores = [
np.dot(hyp_embedding, doc_emb) for doc_emb in doc_embeddings
]
top_indices = np.argsort(scores)[-top_k:][::-1]
return [documents[i] for i in top_indices]
```

HyDE: embed a hypothetical answer instead of the question — matches document vocab better

■ Hybrid Search — BM25 + Semantic Vectors

Neither semantic search (vectors) nor keyword search (BM25) is perfect alone. **Hybrid search** runs both and combines the results using **Reciprocal Rank Fusion (RRF)**.

```
from rank_bm25 import BM25Okapi

# ■■ BM25 keyword retriever ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
tokenized_docs = [doc.split() for doc in documents]
bm25 = BM25Okapi(tokenized_docs)

def bm25_search(query: str, top_k: int = 5) -> list[int]:
"""Keyword-based retrieval (good for exact terms)."""
scores = bm25.get_scores(query.split())
return np.argsort(scores)[-top_k:][::-1].tolist()

def semantic_search(query: str, top_k: int = 5) -> list[int]:
"""Vector-based retrieval (good for meaning/concepts)."""
q_emb = embed(query)
scores = [np.dot(q_emb, d) for d in doc_embeddings]
return np.argsort(scores)[-top_k:][::-1].tolist()

def reciprocal_rank_fusion(ranked_lists: list[list[int]], k: int = 60):
"""Merge multiple ranked lists into a single best list."""
scores = {}
for ranked in ranked_lists:
for rank, doc_id in enumerate(ranked):
scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank + 1)
return sorted(scores, key=scores.get, reverse=True)

def hybrid_search(query: str, top_k: int = 5) -> list[str]:
"""Combine BM25 and semantic search for best coverage."""
bm25_ids = bm25_search(query)
semantic_ids = semantic_search(query)
fused_ids = reciprocal_rank_fusion([bm25_ids, semantic_ids])
return [documents[i] for i in fused_ids[:top_k]]
```

Hybrid search: BM25 catches exact keywords, vectors catch meaning, RRF merges both

■ Multi-Hop Search — Multiple Retrieval Rounds

For complex questions, one round of retrieval isn't enough. **Multi-hop search** feeds retrieved documents back to the LLM, which generates new, more focused queries for subsequent retrieval rounds.

```
class QueryGeneration(dspy.Signature):
"""Generate optimized search queries based on the question and context so
far."""
question: str = dspy.InputField()
retrieved_so_far: str = dspy.InputField(
desc="Documents already retrieved — use these to refine the next
query"
)
semantic_query: str = dspy.OutputField(
desc="Query for vector similarity search"
)
bm25_query: str = dspy.OutputField(
desc="Query for keyword/BM25 search"
)

class MultiHopRetriever(dspy.Module):
def __init__(self, n_hops: int = 3):
self.n_hops = n_hops
self.gen_queries = dspy.ChainOfThought(QueryGeneration)

def forward(self, question: str) -> list[str]:
all_results = []
context_so_far = ""

for hop in range(self.n_hops):
# Generate new queries based on what we've found so far
queries = self.gen_queries(
question=question,
retrieved_so_far=context_so_far
)
# Retrieve using both methods
hop_results = hybrid_search(queries.semantic_query)
all_results.extend(hop_results)
# Update context for next hop
context_so_far = "\n".join(all_results[-6:])

# Deduplicate results
return list(dict.fromkeys(all_results))
```

Multi-hop: each retrieval round informs the next, building deeper understanding

■ Memory — The Mem0 Algorithm

For chatbots, you need **persistent memory** across conversations. The Mem0 approach stores user facts as structured memories and retrieves relevant ones at query time — combining RAG + tool calling.

```
class MemoryOperation(dspy.Signature):
"""Decide what to do with memory based on the new information."""
new_information: str = dspy.InputField()
existing_memories: str = dspy.InputField()
operation: str = dspy.OutputField(
desc="One of: ADD, UPDATE, DELETE, NONE"
)
memory_content: str = dspy.OutputField(
desc="The memory to add/update (empty if DELETE or NONE)"
)

class SimpleMem0:
def __init__(self):
self.memories = [] # List of stored facts
self.memory_embeddings = [] # Their embeddings
self.manager = dspy.Predict(MemoryOperation)

def update(self, new_info: str):
"""Process new info and update memory store accordingly."""
existing = "\n".join(self.memories[-10:]) # Last 10 memories
result = self.manager(
new_information=new_info,
existing_memories=existing
)
if result.operation == "ADD":
self.memories.append(result.memory_content)
self.memory_embeddings.append(embed(result.memory_content))

def recall(self, query: str, top_k: int = 3) -> list[str]:
"""Retrieve the most relevant memories for a query."""
if not self.memories:
return []
q_emb = embed(query)
scores = [np.dot(q_emb, m) for m in self.memory_embeddings]
top_i = np.argsort(scores)[-top_k:][::-1]
return [self.memories[i] for i in top_i]

# Usage
mem = SimpleMem0()
mem.update("The user's name is Sourav and he loves Python.")
mem.update("Sourav prefers concise answers without bullet points.")

relevant = mem.recall("What does the user prefer?")
print(relevant)
# ["Sourav prefers concise answers without bullet points."]
```

Mem0-style memory: LLM decides what to remember, RAG retrieves relevant memories

08 Production Best Practices

1. Design Evaluation First

Before writing code, decide how you'll measure success. Objective metrics (accuracy, ROUGE score) are best. If unavailable, use LLM-as-judge pairwise comparisons.

2. Always Use Structured Outputs

Use dspy.Signature to define typed outputs everywhere. This eliminates JSON parsing errors and gives you reliable programmatic access to each field.

3. Design for Failure

Always ask: what happens when the LLM returns bad output? DSPy auto-retries with corrective prompts, but you should also add fallback logic and input validation.

4. Monitor Token Usage and Latency

LLM costs add up fast. Use MLflow, Langfuse, or Logfire to track token counts, costs, and latency per module so you can optimize where it matters most.

5. Chunk Documents Thoughtfully

When building RAG, generate metadata per chunk (e.g. 'questions this chunk answers'). Store that metadata — it makes retrieval far more accurate.

6. Start Simple, Then Add Complexity

Start with dspy.Predict. Add ChainOfThought if accuracy suffers. Add tools only when needed. Add RAG only when you have external data. Each layer adds latency and cost.

■ The full source code for every example in this guide is available at:
github.com/avbiswas/context-engineering-dspy

★ Quick Reference — DSPy Cheat Sheet

DSPy Component	What It Does	When to Use It
dspy.Signature	Defines input/output contract	Always — every LLM call
dspy.Predict	Basic LLM call with structured output	Simple, single-step tasks
dspy.ChainOfThought	Adds reasoning before final answer	Complex or multi-part questions
dspy.Module	Custom pipeline combining modules	Any multi-step workflow
dspy.ReAct	Reasoning + tool calling agent	When LLM needs external data

DSPy Component	What It Does	When to Use It
set_lm()	Assign a specific LLM to a module	Multi-model routing by task
acall()	Async version of any module call	Parallel generation
BaseModel (Pydantic)	Typed intermediate data structures	Passing structured data between agents

Based on 'DSPy + Context Engineering — Basics to Pro' by Avishek Biswas • youtube.com/watch?v=5Bym0ffALaU • 2025